Design Documentation

Interactive House - AI Smart Control System

Revision History

Date	Version	Description	Author
2026-02-24	1.0	Initial AI design including safety and sensors (D1, D2)	Anitta Antony
2026-02-24	1.0	Initial AI design including safety and sensors (D3, D4)	Md Abu Taher
2026-02-28	1.0	Smart Home code	Daniel Marcarini

Design item List

Requirement Name	Priority
D1. AI Architecture Overview & Component Design	Essential
D2. SIntent Output Format + Prompting Rules(Gemini)	Essential
D3. Mobile AI Chat UI Design (AiScreen.tsx) + API Contract	Essential
D4. Error Handling, Reliability & State/Context Handling	Desirable

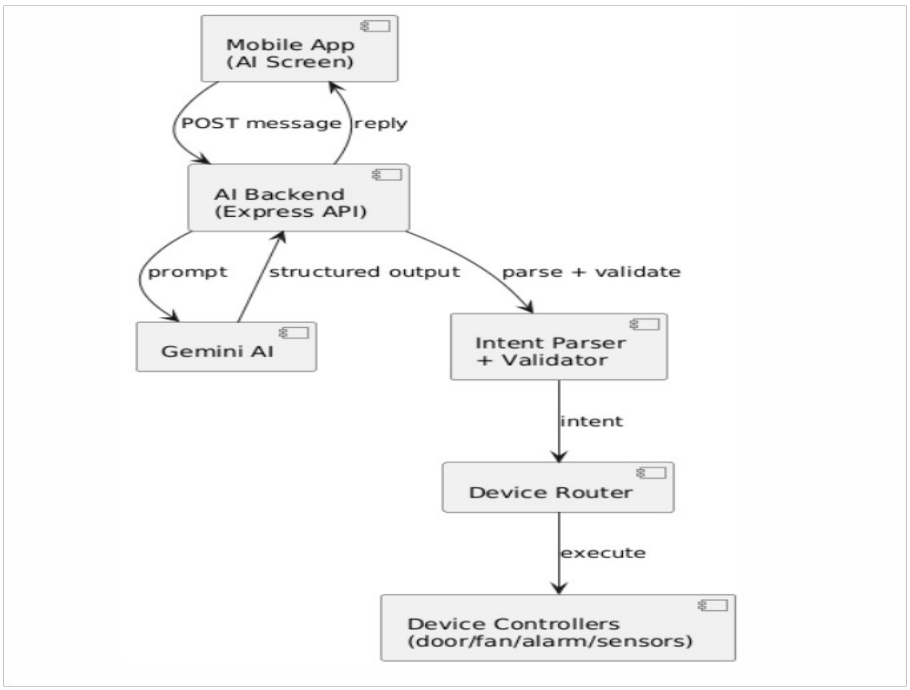
Design Item Descriptions

D1 - AI Architecture Overview & Component Design

This design item defines the overall architecture of the AI feature and how it integrates into the Interactive House project. The AI feature follows a client–server pattern:

- **Mobile App (React Native)** collects the user's natural language message.
- **AI Backend (Node.js/Express)** acts as a secure gateway to the AI model.
- **Gemini API** interprets the message and outputs a structured result.
- **Router/Controller Layer** executes the action (door, ventilation, alarms, sensors).
- The backend sends a response back to the app for display.

This architecture keeps the AI model isolated from direct device access, improving modularity and safety.



D2 - Intent Output Format + Prompting Rules (Gemini)

This design item specifies how Gemini must respond so the backend can execute actions reliably. Natural language is unpredictable, so the AI must output **structured intent**.

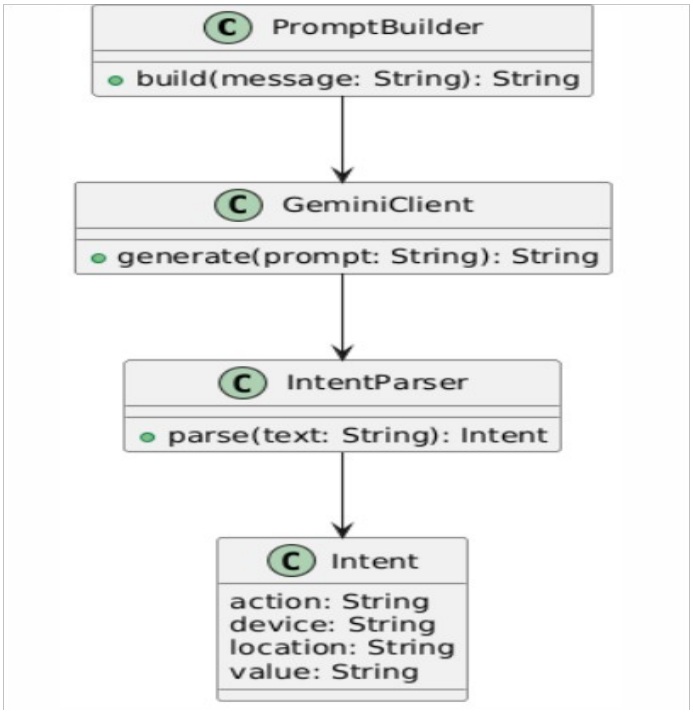
The backend prompt instructs Gemini to return:

1. **Intent JSON** (machine-readable)
2. **User reply text** (human-readable)

Example intent JSON

```
{
 "action": "open",
 "device": "door",
 "location": "main",
 "value": ""
}
```

Figure – Design Class Diagram (AI intent pipeline)



D3 - Mobile AI Chat UI Design (AiScreen.tsx) + API Contract

Description

This design item describes the mobile AI chat interface implemented in `AiScreen.tsx`. The UI enables users to send natural language commands to the backend and receive AI-generated responses.

UI Responsibilities

- Maintain a message list with sender type (`user` / `ai`)
- Provide input field and Send button
- Display loading indicator (“AI is typing…”)
- Automatically scroll to latest message

Use environment variable `EXPO_PUBLIC_API_URL` for backend configuration

API Contract

Endpoint: `POST {BASE_URL}/api/ai-chat`

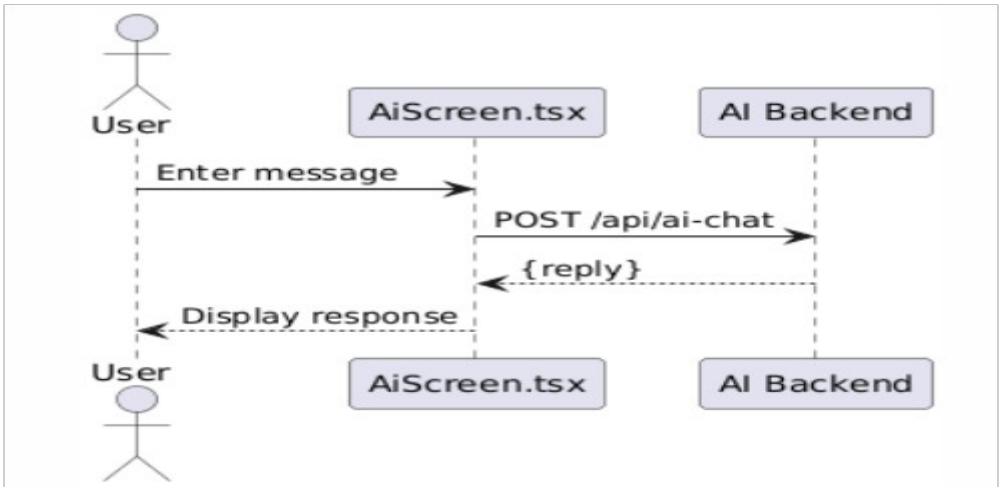
Request Body

{ "message": "Open the door" }

Response Body

{ "reply": "Door opened successfully." }

Figure – Sequence Diagram



D4 - Error Handling and Frontend State Management

Description

This design item explains how the frontend ensures reliability and safe interaction with the AI backend.

Reliability Rules

- Prevent empty message submission
- Prevent duplicate requests while loading
- Display configuration error if `EXPO_PUBLIC_API_URL` is missing
- Show fallback message if backend fails or returns invalid data

Error Handling

- **Missing backend URL:** Display setup warning
- **Backend unreachable:** Show network error message
- **Invalid JSON response:** Handle safely without crashing
- **Non-200 response:** Display returned error message

State Management

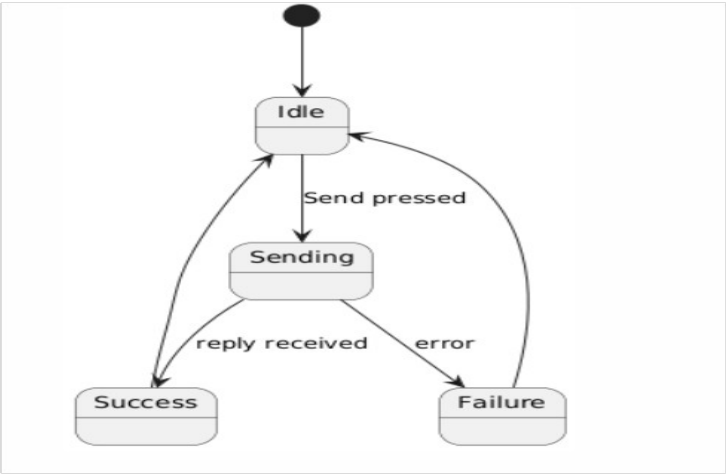
The frontend manages request states using a loading flag:

- Idle
- Sending
- Success
- Failure

This allows the AI system to correctly interpret context-dependent commands such as “Turn it off” by identifying which device is being referenced.

Maintaining device states ensures consistent behavior and improves the intelligence and usability of the AI system.

Figure – State Diagram



Daniel Marcarini - Smart Home code

Requirement Name	Priority
D1. Embedded system architecture and hardware integration	Essential
D2. Sensor monitoring and state machine design	Essential
D3. Actuator control and alarm management system	Essential
D4. LCD display and user interaction handling	Essential
D5. Non-blocking timing and concurrent operation design	Essential

D1 – Embedded System Architecture & Hardware Integration

This design item defines the overall architecture of the embedded Smart Home system and the integration of hardware components.

The system is built around the Arduino Uno microcontroller, which serves as the central processing unit. All sensors, actuators, and interface components are connected directly to the microcontroller.

The architecture separates system functionality into three logical layers:

- Input Layer (gas sensor, temperature sensor, LDR, push buttons, Bluetooth)
- Processing Layer (state management and decision logic)
- Output Layer (servo motors, buzzer, LEDs, LCD display)

This layered design improves clarity and modularity while maintaining simplicity suitable for embedded systems.

D2 – Sensor Monitoring and State Machine Design

This design item specifies how environmental data is processed and how system states are managed.

The system continuously monitors:

- Gas levels
- Temperature
- Ambient light

Gas detection is implemented using a threshold-based comparison combined with a staged state machine. The state machine ensures that safety actions are executed in a controlled sequence rather than simultaneously.

Internal state variables maintain:

- Window open/closed state
- Door open/closed state
- Alarm activation status
- Gas detection transition state

This prevents repeated triggering and ensures stable system behavior.

D3 – Actuator Control and Alarm Management System

This design item describes how mechanical and alarm components are controlled.

The system controls:

- Window servo motor
- Door servo motor
- Buzzer (multiple modes)
- LED indicators

Buzzer behavior is centralized using defined operational modes (e.g., off, solid alarm, siren mode). This prevents conflicting sound activation from different subsystems.

During emergency events:

- The buzzer is activated
- The window may open automatically
- Visual indicators reflect hazard status

Safety-related actions override normal user interaction.

D4 – LCD Display and User Interaction Handling

This design item defines how the system provides feedback and processes user input.

The LCD module displays:

- Environmental readings
- Alarm notifications
- Device status information

Display updates are controlled to prevent flickering and unnecessary refresh operations.

User interaction is implemented using:

- Push buttons with edge-detection logic
- Bluetooth communication for remote commands

Button presses are detected through state comparison to ensure reliable toggling behavior.

D5 – Non-Blocking Timing and Concurrent Operation Design

This design item explains how the system maintains real-time responsiveness.

The system uses non-blocking timing mechanisms based on internal timers instead of delay-based blocking. This allows:

- Continuous sensor monitoring
- Immediate emergency response
- Simultaneous user interaction handling
- Stable LCD updates

This design ensures that no subsystem prevents another from executing, maintaining reliable concurrent behavior within the embedded environment.

D6 – Speech Recognition Integration and Voice Navigation Design

- **Description**

This design item describes the integration of speech recognition into the mobile application using Expo and an external speech recognition library. The feature allows users to control the Interactive House system using spoken commands instead of manual text input.

- **Technology and Build Configuration**

The speech recognition functionality requires native device capabilities that are not available in the standard Expo Go environment. Therefore, a custom Expo development build was created to support the external speech recognition library and enable microphone access and real-time voice processing.

- **Architecture Integration**

The speech recognition module is implemented as a dedicated screen (tab) within the mobile application. The module interacts with the existing navigation and AI chat system.

The interaction flow is:

1. The user opens the Speech Recognition tab.

2. The user activates the microphone using the UI control.

3. Spoken input is captured and converted into text using the speech recognition library.

4. The recognized text is displayed and forwarded to the backend AI API. (Future feature)

5. The backend processes the command and returns a response.(Future feature)

The application navigates to the relevant screen or updates the system state based on the interpreted command.

- **Navigation Integration**

Speech recognition can trigger navigation between different screens in the application. This allows users to move between system features using voice commands, improving accessibility and interaction efficiency.

- **Future Extension – Floating Voice Control Module**

The system architecture is designed to support a future floating voice control component. This component will allow speech interaction from any screen without requiring the user to manually open the speech tab. The floating module will act as a global input interface integrated with the existing AI backend and navigation system.